Visvesvaraya Technological University, Belagavi – 590018.

PROJECT PHASE-I REPORT
ON

Intelligent Academic-to-Industry Transition Support System Using Machine Learning

Submitted in partial fulfillment for the award of degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shivaraj	**4CB23CS141**
Sridharshan	**4CB23CS160**
Suhas N R	**4CB23CS161**
Vishwas H Acharya	**4CB23CS187**

Under the Guidance of

Ms. Sushma
Assistant Professor, Department of CS&E

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CANARA ENGINEERING COLLEGE

(An Autonomous Institution, Under VTU Belagavi, Recognized by AICTE, Accredited by NBA (CSE, ISE, ECE) and NAAC 'A' GRADE)

Sudhindra Nagar, Benjanapadavu, Mangaluru - 574219, Karnataka.

A.Y 2025-26

CANARA ENGINEERING COLLEGE

(An Autonomous Institution, Under VTU Belagavi, Recognized by AICTE, Accredited by NBA (CSE, ISE, ECE) and NAAC 'A' GRADE)

Sudhindra Nagar, Benjanapadavu, Mangaluru - 574219, Karnataka.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the Project work Phase-I entitled **"Intelligent Academic-to-Industry Transition Support System Using Machine Learning"** carried out by

Shivaraj	**4CB23CS141**
Sridharshan	**4CB23CS160**
Suhas N R	**4CB23CS161**
Vishwas H Acharya	**4CB23CS187**

the bonafide students of VI semester COMPUTER SCIENCE AND ENGINEERING in partial fulfillment for the award of Bachelor of Engineering in COMPUTER SCIENCE AND ENGINEERING of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions given during internal reviews are included for Internal Assessment. The Project Phase-I report has been approved as it satisfies the academic requirements in respect of Phase-I Project work prescribed for the said degree.

Ms. Sushma	**Dr. Vinay P**	**Dr. Karthik Pai B H**
Project Guide	Project-Coordinator	HoD

Dr. Demian Antony D'Mello	**Principal**
Dean-Academics	Canara Engineering College

Internal / External Viva:

Examiner's Name Signature with Date

1. .....
.....

2. .....
.....

CANARA ENGINEERING COLLEGE

(An Autonomous Institution, Under VTU Belagavi, Recognized by AICTE, Accredited by NBA (CSE, ISE, ECE) and NAAC 'A' GRADE)

Sudhindra Nagar, Benjanapadavu, Mangaluru - 574219, Karnataka.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We hereby declare that our entire work embodied in this Project Report titled **"Intelligent Academic-to-Industry Transition Support System Using Machine Learning"** has been carried out at CANARA ENGINEERING COLLEGE, Mangaluru under the supervision of **Ms. Sushma**, Assistant Professor, Department of Computer Science and Engineering, for the award of **Bachelor of Engineering** in **COMPUTER SCIENCE AND ENGINEERING**. This report has not been submitted to this or any other University for the award of any other degree.

Shivaraj (4CB23CS141) .....

Sridharshan (4CB23CS160) .....

Suhas N R (4CB23CS161) .....

Vishwas H Acharya (4CB23CS187) .....

Acknowledgement

We dedicate this page to acknowledge and thank those responsible for the shaping of this project. Without their constant guidance, valuable critique, and encouragement, constructing this project report would not have been so smooth and efficient.

We sincerely thank our Project Guide **Ms. Sushma**, Assistant Professor, Department of Computer Science and Engineering, for her keen guidance, constructive suggestions, and continuous supervision which helped us navigate technical roadblocks and complete this phase successfully.

We also convey our sincere gratitude to our Project Coordinator **Dr. Vinay P**, Department of Computer Science and Engineering, for his consistent encouragement, milestone scheduling, and insightful reviews during internal assessments.

We owe a profound gratitude to **Dr. Karthik Pai B H**, Head of the Department, Computer Science and Engineering, whose constant encouragement, administrative support, and academic guidance enabled us to work in an inspiring environment.

We also take this opportunity to thank our Dean Academics **Dr. Demian Antony D'Mello** and our respected Principal for providing state-of-the-art infrastructural facilities and computational resources to carry out our project work.

We express our gratitude to all teaching and non-teaching staff of the Department of Computer Science and Engineering for their direct and indirect support throughout the course of this work. Finally, we convey our deepest appreciation to our parents and friends for their enduring patience, moral support, and enthusiasm.

Shivaraj (4CB23CS141)
Sridharshan (4CB23CS160)
Suhas N R (4CB23CS161)
Vishwas H Acharya (4CB23CS187)

Abstract

Graduating engineering students frequently encounter severe hurdles while transitioning from academic environments to industry workplaces due to discrepancies between standard academic curricula and fast-evolving corporate skill requirements. Traditional campus placement preparation relies on generic guidance, manual resume reviews, and isolated assessment tests, which fail to pinpoint individual technical and soft-skill deficiencies.

To overcome these challenges, this project presents the **Intelligent Academic-to-Industry Transition Support System Using Machine Learning**, an end-to-end, production-ready platform designed to provide automated resume parsing, NLP-based skill extraction, machine learning-driven employability prediction, comprehensive gap analysis, and tailored job and course recommendations. The system utilizes an automated parsing engine capable of processing unstructured documents (PDF, DOCX, TXT) into structured profiles using regular expressions and resilient Transformer-based NLP pipelines (BERT/DistilBERT). Employability readiness is predicted using an ensemble Stacking Classifier combining Random Forest, Gradient Boosting, and Support Vector Machines with a Logistic Regression meta-classifier, alongside a continuous 6-pillar calibrated employability scoring model evaluating academic performance, technical skills, practical experience, internships, and certifications.

The backend is developed in Python Flask adhering to modern REST standards with JWT authentication, multi-level caching, and defensive anomaly detection via Isolation Forests and Interquartile Range methods. The user interface is engineered as a responsive Single Page Application using React and Vite. Experimental evaluations on student cohorts demonstrate that the ensemble model achieves a classification accuracy of 75.50% with an F1 score of 0.80, substantially outperforming single-model baselines and providing actionable, personalized career pathways for undergraduate students.

Keywords: Machine Learning, Natural Language Processing, Resume Parsing, Employability Prediction, Skill Gap Analysis, Stacking Ensemble, Recommendation System, BERT.

Table of Contents

Acknowledgement i

Abstract ii

Table of Contents vi

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Background . . . 1
1.2 Motivation and Problem Statement . . . 1
1.2.1 Motivation . . . 1
1.2.2 Problem Statement . . . 2
1.3 Objectives . . . 2
1.4 Scope and Limitations . . . 3
1.4.1 Scope . . . 3
1.4.2 Limitations . . . 3
1.5 Relevance and Type . . . 3
1.6 Organization of the Report . . . 3
1.7 Summary . . . 4

2 Literature Survey 5

2.1 Skill-Driven Certification Pathways: Measuring Industry Training Impact on Graduate Employability . . . 5
2.1.1 Brief Findings of Article 1 . . . 5
2.1.2 Methodology Adopted in Article 1 . . . 5
2.1.3 Results Achieved in Article 1 . . . 5
2.2 Rapid Industry Skill Adoption Framework (RISAF) . . . 5
2.2.1 Brief Findings of Article 2 . . . 5
2.2.2 Methodology Adopted in Article 2 . . . 6
2.2.3 Results Achieved in Article 2 . . . 6
2.3 Predicting Post-Internship Employability Using Ensemble Machine Learning 6
2.3.1 Brief Findings of Article 3 . . . 6
2.3.2 Methodology Adopted in Article 3 . . . 6
2.3.3 Results Achieved in Article 3 . . . 6

2.4 Career Prediction Application . . . 6
2.4.1 Brief Findings of Article 4 . . . 6
2.4.2 Methodology Adopted in Article 4 . . . 6
2.4.3 Results Achieved in Article 4 . . . 7
2.5 CareerCraft AI: Intelligent Career Advisory Systems . . . 7
2.5.1 Brief Findings of Article 5 . . . 7
2.5.2 Methodology Adopted in Article 5 . . . 7
2.5.3 Results Achieved in Article 5 . . . 7
2.6 Employability Prediction of IT Graduates Using Machine Learning . . . 7
2.6.1 Brief Findings of Article 6 . . . 7
2.6.2 Methodology Adopted in Article 6 . . . 7
2.6.3 Results Achieved in Article 6 . . . 7
2.7 Machine Learning Classification Techniques for Graduate Employability . 8
2.7.1 Results Achieved in Article 7 . . . 8
2.8 Skill Gap Analysis Using Data Mining Techniques . . . 8
2.8.1 Results Achieved in Article 8 . . . 8
2.9 Hybrid Career Recommendation System . . . 8
2.9.1 Results Achieved in Article 9 . . . 8
2.10 AI-Based Student Skill Assessment and Recommendation System . . . 8
2.10.1 Results Achieved in Article 10 . . . 8
2.11 Comparison Table . . . 9
2.12 Gap's Identified and Addressed . . . 9
2.13 Summary . . . 9

3 Software Requirements Specification 10
3.1 Functional Requirements . . . 10
3.2 Non-Functional Requirements . . . 11
3.2.1 Safety and Reliability Requirements . . . 11
3.2.2 Performance Requirements . . . 11
3.2.3 Security Requirements . . . 12
3.3 User Interface Design . . . 12
3.4 Hardware and Software Requirements . . . 12
3.4.1 Hardware Requirements . . . 12
3.4.2 Software Requirements . . . 13
3.5 Summary . . . 13

4 System Design 14
4.1 Abstract Design . . . 14
4.1.1 Architectural Layer Descriptions . . . 14
4.2 Proposed System Flow . . . 14
4.3 Functional Design . . . 15

4.3.1 Modular Design . . . 15
4.3.2 Sequence Diagram . . . 16
4.3.3 Use Case Diagram . . . 17
4.4 Control Flow Design . . . 18
4.4.1 Activity Diagram . . . 18
4.5 Data Flow Design . . . 19
4.5.1 Zero-Level Data Flow Diagram (Level 0 DFD) . . . 19
4.5.2 First-Level Data Flow Diagram (Level 1 DFD) . . . 19
4.5.3 Second-Level Data Flow Diagram (Level 2 DFD) . . . 20
4.6 Summary . . . 20

5 Implementation **21**
5.1 Software Used with Justification . . . 21
5.1.1 Frontend Development . . . 21
5.1.2 Backend Development . . . 21
5.1.3 Machine Learning and Data Engineering Libraries . . . 21
5.1.4 Operating System and Environment . . . 21
5.2 Hardware Used with Justification . . . 22
5.3 Algorithms and Procedures Used . . . 22
5.3.1 Algorithm 1: NLP Resume Parsing and Skill Extraction . . . 22
5.3.2 Algorithm 2: Stacking Ensemble Employability Classifier . . . 22
5.3.3 Algorithm 3: 6-Pillar Calibrated Employability Regressor . . . 23
5.3.4 Algorithm 4: Vector Space Skill Matching and Gap Remediation . 23
5.4 Coding and Software Engineering Practices . . . 23
5.4.1 Introduction . . . 23
5.4.2 Programming Style and Standards . . . 23
5.5 Summary . . . 24

6 Results and Discussion **25**
6.1 Experimental Results . . . 25
6.2 Software Testing . . . 26
6.3 Types of Software Testing . . . 26
6.4 Testing Methodology . . . 26
6.4.1 Unit Testing . . . 26
6.4.2 Integration Testing . . . 26
6.4.3 System Testing . . . 26
6.4.4 Field Testing . . . 26
6.4.5 Acceptance Testing . . . 27
6.5 Testing Criteria and Test Cases . . . 28
6.5.1 Testing for User Authentication and Inputs . . . 28
6.5.2 Testing for Resume Upload and Data Insertion . . . 28

6.5.3 Testing for Deletion and Record Removal 28
6.5.4 Testing for Filter Criteria and Date Durations 29
6.5.5 Other Comprehensive Test Cases 29
6.6 Comparative Analysis of Machine Learning Models 29
6.7 Summary 30

7 Conclusions and Future work **31**

References **32**

A Drill-Bit / Turnitin Plagiarism Report **32**

B Patent / Publication and Project-Expo Details **33**

List of Figures

4.1 System Architecture Diagram 14
4.2 Proposed System Operational Flowchart 14
4.3 Modular Design Diagram 15
4.4 System Sequence Diagram 16
4.5 Use Case Diagram 17
4.6 Activity Diagram for Resume Evaluation Workflow 18
4.7 Zero-Level Data Flow Diagram (Context Diagram) 19
4.8 First-Level Data Flow Diagram 19
4.9 Second-Level Data Flow Diagram (ML & Parsing Subsystem) 20

6.1 User Authentication and Dashboard Interface 25
6.2 Employability Prediction, Gap Analysis, and Recommendations Outcome 25

List of Tables

2.1 Comparison of Existing Work and Gap Identification 9

6.1 Test cases for user registration and authentication 28
6.2 Test cases for resume parsing and data ingestion 28
6.3 Test cases for record deletion and session teardown 28
6.4 Test cases for date ranges and filtering 29
6.5 Other functional and UI test cases 29
6.6 Comparative performance analysis of classification models 29

Introduction

1.1 Background

In contemporary higher education, the rapid advancement of software engineering methodologies, cloud computing, artificial intelligence, and DevOps has created a significant divide between academic syllabi and industrial competence expectations [**zhang2021deep**]. While undergraduate curricula provide fundamental theoretical knowledge in mathematics, algorithms, and data structures, modern industrial recruiters demand proficiency in specialized programming frameworks, practical project architectures, containerization, and continuous integration workflows. Consequently, a large portion of graduating engineers struggle to secure suitable entry-level roles during campus recruitment drives [**kumar2023ensemble**].

The process of bridging this gap necessitates continuous, personalized evaluation of student technical competence against live corporate benchmarks. However, conventional campus placement cells rely heavily on manual counseling, static spreadsheet tracking, and periodic aptitude assessments that fail to provide real-time, data-driven feedback [**rao2021visionary**]. With thousands of students graduating annually, institutional placement officers lack automated systems to parse diverse student resumes, diagnose granular skill gaps, predict placement probabilities, and suggest targeted learning roadmaps.

1.2 Motivation and Problem Statement

1.2.1 Motivation

With the digital transformation of technical recruitment, automated Applicant Tracking Systems (ATS) reject over 70% of candidate resumes before human recruiters even inspect them. Engineering students often possess relevant domain knowledge but fail to structure their resumes effectively or lack key prerequisite keywords demanded by industry job descriptions. Providing students with an intelligent, self-service platform that analyzes their resume, assesses their employability readiness score, highlights missing domain skills, and provides structured course and job recommendations empowers them to take proactive corrective action before attending placement drives.

1.2.2 Problem Statement

Existing student evaluation and recruitment support solutions exhibit three primary shortcomings:

1. **Absence of Automated Multi-Format Resume Intelligence**: Inability to accurately ingest, clean, and extract technical entities from unstructured multi-format resumes (PDF, DOCX, TXT) without manual data entry.
2. **Heuristic vs. Probabilistic Employability Assessment**: Reliance on single-metric academic thresholds (e.g., CGPA alone) rather than a holistic multi-dimensional evaluation covering projects, certifications, internships, skill diversity, and academic consistency.
3. **Disconnected Career Guidance**: Career recommendations are often generic and disconnected from market job requirements, lacking actionable course links and skill-matching similarity metrics.

1.3 Objectives

The primary objective of this project is to architect, develop, and validate an end-to-end intelligent platform that facilitates a seamless academic-to-industry transition for engineering students. The specific sub-objectives are:

1. To design a robust, fault-tolerant document ingestion engine capable of parsing resumes in PDF, DOCX, and TXT formats using regex, keyword dictionaries, and Transformer NLP models (BERT/DistilBERT).
2. To implement a feature engineering and data processing pipeline that extracts 13+ structural indicators including CGPA normalization, skill diversity, internship duration, project scores, and experience ratios.
3. To construct an ensemble Stacking Classifier (Random Forest + Gradient Boosting + Support Vector Machine meta-learned by Logistic Regression) and a 6-pillar calibrated regressor to accurately predict student placement likelihood.
4. To build an automated recommendation subsystem using Vector Space Models and Cosine Similarity to compare extracted candidate skill profiles with dynamic industry job postings and online course catalogs.
5. To engineer a secure, modern, and responsive full-stack web application with JWT authentication, multi-tier caching, automated anomaly detection, and interactive visual dashboards.

1.4 Scope and Limitations

1.4.1 Scope

The scope of this project encompasses undergraduate engineering disciplines (with primary calibration for Computer Science and Engineering and allied branches). The system handles end-to-end workflows including user registration, authenticated resume upload, automated parsing, machine learning inference, skill gap visualization, and real-time job/course mapping. The architecture is engineered to run seamlessly across both cloud-hosted environments and resource-constrained institutional servers.

1.4.2 Limitations

- **Graphic/Scanned Resumes**: The parser relies on optical character and programmatic text extraction; highly artistic image-based resumes without OCR layers may require pre-processing.
- **Evolving Technology Taxonomies**: Newly emerging programming frameworks require periodic updates to the central skill dictionary or transformer fine-tuning.
- **Institutional Data Privacy**: Direct automated synchronization with proprietary university grade portals requires specific API integrations per university.

1.5 Relevance and Type

This project is an applied interdisciplinary work spanning **Machine Learning, Natural Language Processing, Data Engineering, and Full-Stack Web Development**. It directly addresses the objectives of outcome-based education (OBE) under the National Education Policy (NEP 2020) and Visvesvaraya Technological University (VTU) guidelines by introducing quantifiable, data-backed interventions to enhance graduate employability.

1.6 Organization of the Report

The remainder of this report is structured as follows:

- **Chapter 2 (Literature Survey)**: Reviews state-of-the-art research papers in automated resume parsing, employability prediction, and recommender systems, concluding with a comparative analysis table and gap identification.
- **Chapter 3 (Software Requirements Specification)**: Details the functional, non-functional, hardware, software, and interface requirements of the platform.

- **Chapter 4 (System Design)**: Outlines the system architecture, modular design, sequence diagrams, use case interactions, activity flows, and multi-level data flow diagrams (Level 0, 1, and 2 DFDs).
- **Chapter 5 (Implementation)**: Explains the software tools, mathematical algorithms, machine learning ensemble design, NLP pipeline, and software engineering principles adopted.
- **Chapter 6 (Results and Discussion)**: Presents experimental findings, model evaluation metrics, UI screenshots, comprehensive testing methodologies, and structured test case tables.
- **Chapter 7 (Conclusions and Future Work)**: Summarizes the project contributions and highlights avenues for future enhancements.

1.7 Summary

This chapter established the foundational motivation, problem definition, objectives, and organizational structure of the report. The next chapter presents a comprehensive literature survey analyzing contemporary works in the domain of automated recruitment analytics and student career readiness.

Literature Survey

2.1 Skill-Driven Certification Pathways: Measuring Industry Training Impact on Graduate Employability

[**kovalev2025skill**]

This study analyzes over 2.5 million job advertisements from Australia, the United States, and the United Kingdom to understand how certifications influence graduate employability.

2.1.1 Brief Findings of Article 1

The authors demonstrate that certifications significantly improve alignment between graduate skills and job requirements. In some cases, the combination of academic degrees and certifications dramatically increases employment opportunities.

2.1.2 Methodology Adopted in Article 1

The study uses the Vectorised Skills Space Method (VSSM), SkillNER, and a skill similarity matrix to quantify relationships between certifications and occupations.

2.1.3 Results Achieved in Article 1

Certifications such as Microsoft AI-900 were shown to substantially improve employability.

2.2 Rapid Industry Skill Adoption Framework (RISAF)

[**weerasuriya2026rapid**]

RISAF is a conceptual framework designed to help final-year IT students adapt quickly to industry environments.

2.2.1 Brief Findings of Article 2

The paper highlights deficiencies in workflow exposure, tooling knowledge, and real-world problem-solving.

2.2.2 Methodology Adopted in Article 2

The framework includes Reality Alignment, Guided Micro Projects, and Industry Thinking Development.

2.2.3 Results Achieved in Article 2

Provides a practical transition model, but lacks experimental validation.

2.3 Predicting Post-Internship Employability Using Ensemble Machine Learning

[**kahlik2024predicting**]

This study predicts student employability based on internship-related factors.

2.3.1 Brief Findings of Article 3

Internship performance, duration, and training quality are strong indicators of employability.

2.3.2 Methodology Adopted in Article 3

KNN, SVM, XGBoost, and stacking ensemble models were evaluated.

2.3.3 Results Achieved in Article 3

The stacking model achieved approximately 91% accuracy.

2.4 Career Prediction Application

[**garje2026career**]

This application maps student skills and interests to career roles.

2.4.1 Brief Findings of Article 4

Provides confidence scores and readiness levels.

2.4.2 Methodology Adopted in Article 4

Weighted Scoring Model with Flask and MySQL.

2.4.3 Results Achieved in Article 4

Improved decision-making, but remained largely rule-based.

2.5 CareerCraft AI: Intelligent Career Advisory Systems

[**talekar2025careercraft**]

CareerCraft AI is an intelligent career advisory system using Machine Learning and NLP.

2.5.1 Brief Findings of Article 5

Supports resume analysis, skill gap detection, and personalized recommendations.

2.5.2 Methodology Adopted in Article 5

Random Forest, TF-IDF, and collaborative filtering.

2.5.3 Results Achieved in Article 5

Generated customized career paths and recommendations.

2.6 Employability Prediction of IT Graduates Using Machine Learning

[**elsharkawy2022employability**]

The study uses academic and skill data to classify employability.

2.6.1 Brief Findings of Article 6

Academic performance and technical skills are major predictors.

2.6.2 Methodology Adopted in Article 6

Decision Tree, Logistic Regression, and SVM.

2.6.3 Results Achieved in Article 6

Accuracy reached 95–98%.

2.7 Machine Learning Classification Techniques for Graduate Employability

[**haque2024classification**]
This work compares ANN, Naive Bayes, and SVM.

2.7.1 Results Achieved in Article 7

Artificial Neural Networks performed best.

2.8 Skill Gap Analysis Using Data Mining Techniques

[**sharma2021skill**]
This study compares job descriptions and student skills.

2.8.1 Results Achieved in Article 8

Clustering and association rules identified critical missing skills.

2.9 Hybrid Career Recommendation System

[**patel2023hybrid**]
Combines collaborative filtering with Machine Learning.

2.9.1 Results Achieved in Article 9

Improved recommendation accuracy and personalization.

2.10 AI-Based Student Skill Assessment and Recommendation System

[**verma2024ai**]
Evaluates students and recommends improvement strategies.

2.10.1 Results Achieved in Article 10

Produced detailed skill reports and personalized suggestions.

2.11 Comparison Table

Table 2.1: Comparison of Existing Work and Gap Identification

Paper	**Key Contribution**	**Methodology**	**Limitation**
Skill-Driven Certification Pathways	Large-scale skill and certification analysis	VSSM, SkillNER	No integrated student platform
RISAF	Structured transition framework	Conceptual framework	Conceptual only
Post-Internship Employability	High prediction accuracy	Ensemble ML	Focuses only on internship data
Career Prediction Application	Readiness scoring	Weighted Scoring Model	Rule-based model
CareerCraft AI	Resume analysis and recommendations	Random Forest, TF-IDF	Limited validation
Employability Prediction	Very high accuracy	Decision Tree, SVM	No job market integration
Skill Gap Analysis	Missing skill detection	Clustering	Static analysis only
Hybrid Recommendation	Personalized suggestions	Collaborative filtering	Requires large datasets
AI-Based Skill Assessment	Automated evaluation	ML models	No live job integration

2.12 Gap's Identified and Addressed

From the literature survey, the following research gaps were identified:

- Most systems focus on only one functionality.
- Few systems perform direct resume-to-job matching.
- Structured learning path generation is rarely available.
- Faculty monitoring and institutional analytics are missing.
- Real-time job market integration is limited.
- Continuous progress tracking is not commonly supported.

The proposed system addresses these limitations by integrating resume analysis, skill gap detection, employability prediction, course recommendations, assessments, progress tracking, and faculty dashboards in a single platform.

2.13 Summary

This chapter reviewed ten relevant research works and identified the need for a comprehensive intelligent platform to support students throughout their transition from academics to industry.

Software Requirements Specification

3.1 Functional Requirements

Functional requirements define the core operational behaviors and services provided by the system. The platform encompasses five major functional modules:

a) User Authentication and Profile Management

- The system shall allow students to register with their name, email, department, semester, and password.
- The system shall securely authenticate users via JSON Web Tokens (JWT) and enforce password hashing via Bcrypt.
- The system shall maintain user profile information, academic records (CGPA, semester), and past analysis histories in a relational database.

b) Multi-Format Resume Parsing Engine

- The system shall allow users to upload resumes in PDF, DOCX, and plain text (TXT) formats with file size validation ($\leq$ 10 MB).
- The system shall extract text using specialized programmatic extractors (`PyPDF2`, `python-docx`).
- The system shall extract technical skills, programming languages, frameworks, education credentials, project descriptions, and internship experience using regex patterns and NLP keyword matching with offline BERT fallback.

c) ML Employability Prediction Engine

- The system shall extract 13+ normalized features from the parsed resume and student profile.
- The system shall predict the binary placement readiness class (Ready / Needs Improvement) using a pre-trained Stacking Ensemble Classifier.
- The system shall compute a continuous 6-pillar calibrated employability score (30.0−98.0) assessing Skills, Projects, CGPA, Internships, Certifications, and Workshops.

d) Skill Gap Analysis and Recommendation Subsystem

- The system shall compare extracted candidate skills against target industry role taxonomies (e.g., Software Engineer, Data Scientist, DevOps Engineer).
- The system shall identify missing critical skills and calculate a role-matching percentage using Cosine Similarity.
- The system shall retrieve tailored MOOC courses and curated job postings aligned with the candidate's diagnosed skill deficiencies.

e) Data Pipeline and Anomaly Detection Engine

- The system shall clean, normalize, and partition industry job records and student placement datasets.
- The system shall run automated quality checks (completeness, accuracy, validity) and detect outliers using Isolation Forests and Interquartile Range (IQR) methods.

3.2 Non-Functional Requirements

3.2.1 Safety and Reliability Requirements

- **Fault Tolerance**: The system incorporates circuit breakers and automatic retry decorators (max_retries = 3) on network and data services to prevent application crashes from transient errors.
- **Data Backup and Recovery**: The system supports automated SQLite/PostgreSQL database snapshotting and backup recovery routines in the event of hardware failures.

3.2.2 Performance Requirements

- **Inference Latency**: The machine learning prediction and recommendation response time shall not exceed 100 milliseconds for pre-parsed feature vectors.
- **Document Parsing Throughput**: Resume parsing and feature extraction shall complete within 3 seconds for standard multi-page documents.
- **Caching Efficiency**: Multi-level in-memory and Redis caching shall ensure frequent job/course lookups achieve response times under 20 milliseconds.

3.2.3 Security Requirements

- All API endpoints modifying or reading private student data must require a valid Bearer JWT header.
- Passwords must be hashed using one-way cryptographic algorithms with salt (Bcrypt).
- The API layer must enforce rate limiting (via `Flask-Limiter`) to prevent denial-of-service and brute-force attacks.

3.3 User Interface Design

The frontend user interface is designed as a responsive Single Page Application (SPA) following modern design principles:

- **Dashboard Overview**: Displays real-time employability gauge charts, skill distribution radar plots, and progress bars.
- **Resume Upload Dropzone**: Interactive drag-and-drop file uploader with real-time file validation and parsing progress indicators.
- **Skill Gap Matrix**: Side-by-side comparative table contrasting candidate skills against target industrial roles with color-coded tags.
- **Course & Job Carousel**: Interactive cards linking directly to recommended learning materials and open positions.

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

- **Processor**: Intel Core i5 / AMD Ryzen 5 or higher (minimum 2.0 GHz).
- **RAM**: Minimum 8 GB (16 GB recommended for local Transformer model execution).
- **Storage**: Minimum 20 GB available solid-state storage (SSD).
- **Client Device**: Any PC, laptop, or tablet with a modern web browser (Chrome, Edge, Firefox).

3.4.2 Software Requirements

- **Operating System**: Windows 10/11, Ubuntu Linux 20.04+, or macOS.
- **Backend Runtime**: Python 3.11+.
- **Backend Framework**: Flask 3.0, Flask-JWT-Extended, SQLAlchemy 2.0.
- **Machine Learning & NLP**: Scikit-learn 1.3+, PyTorch, HuggingFace Transformers, PyPDF2, python-docx.
- **Frontend Runtime**: Node.js 18+, npm 9+.
- **Frontend Framework**: React 18, Vite, Redux Toolkit, Tailwind CSS / Vanilla CSS.
- **Database**: SQLite (development) / PostgreSQL (production).

3.5 Summary

This chapter detailed the complete functional and non-functional specifications, interface designs, and minimum environment requirements for the platform. The subsequent chapter discusses the architectural and system design representations.

System Design

4.1 Abstract Design

The system follows a decoupled, 3-Tier Client-Server Architecture comprising the Presentation Layer, Application & API Layer, and the Data & Machine Learning Layer.

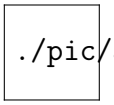


Figure 4.1: System Architecture Diagram

4.1.1 Architectural Layer Descriptions

1. **Presentation Layer (Frontend)**: Built with React 18 and Vite, managing user interactions, authentication state, file upload events, and dynamic Chart.js/Recharts data visualizations.
2. **Application Layer (Flask REST API)**: Modular Flask Blueprints handling route dispatching, JWT authorization, rate limiting, request validation, and orchestrating service calls.
3. **Data & Intelligence Layer**: Houses the NLP parsing engine, Stacking Ensemble classifier, 6-pillar regressor, Vector Space recommendation engine, and relational databases.

4.2 Proposed System Flow

The end-to-end procedural flow of the proposed transition support platform is illustrated below:

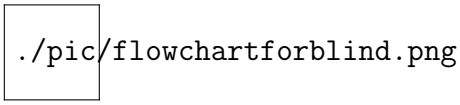


Figure 4.2: Proposed System Operational Flowchart

4.3 Functional Design

4.3.1 Modular Design

The system is partitioned into independent, loosely coupled modules ensuring high cohesion:

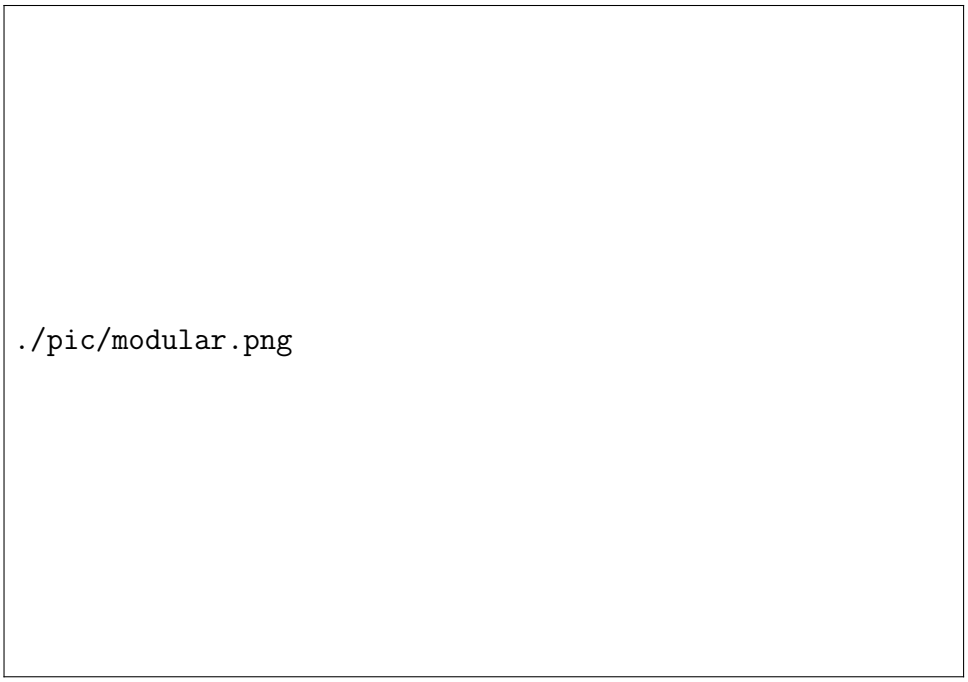


Figure 4.3: Modular Design Diagram

- **Authentication Module**: Manages student login, registration, and cryptographic token issuance.
- **Document Parsing Module**: Ingests resumes, extracts raw text, cleans formatting, and isolates structured entities.
- **ML Scoring Module**: Featurizes candidate data and executes ensemble inference.
- **Gap & Recommendation Module**: Computes cosine similarity against industry role vectors and MOOC databases.
- **Data Pipeline Module**: Performs background ETL, data quality assertion, and anomaly detection.

4.3.2 Sequence Diagram

The sequence diagram models the chronological interaction between system entities during a typical analysis cycle:



Figure 4.4: System Sequence Diagram

As shown, the student uploads their resume through the React SPA. The file is transmitted via a secure multipart REST request to the Flask backend. The parsing engine processes the document, passes extracted attributes to the ML feature extractor, executes Stacking Classifier inference, and generates job/course recommendations before returning a consolidated JSON response to update the frontend dashboard.

4.3.3 Use Case Diagram

The use case diagram illustrates user and administrator interactions with the system:

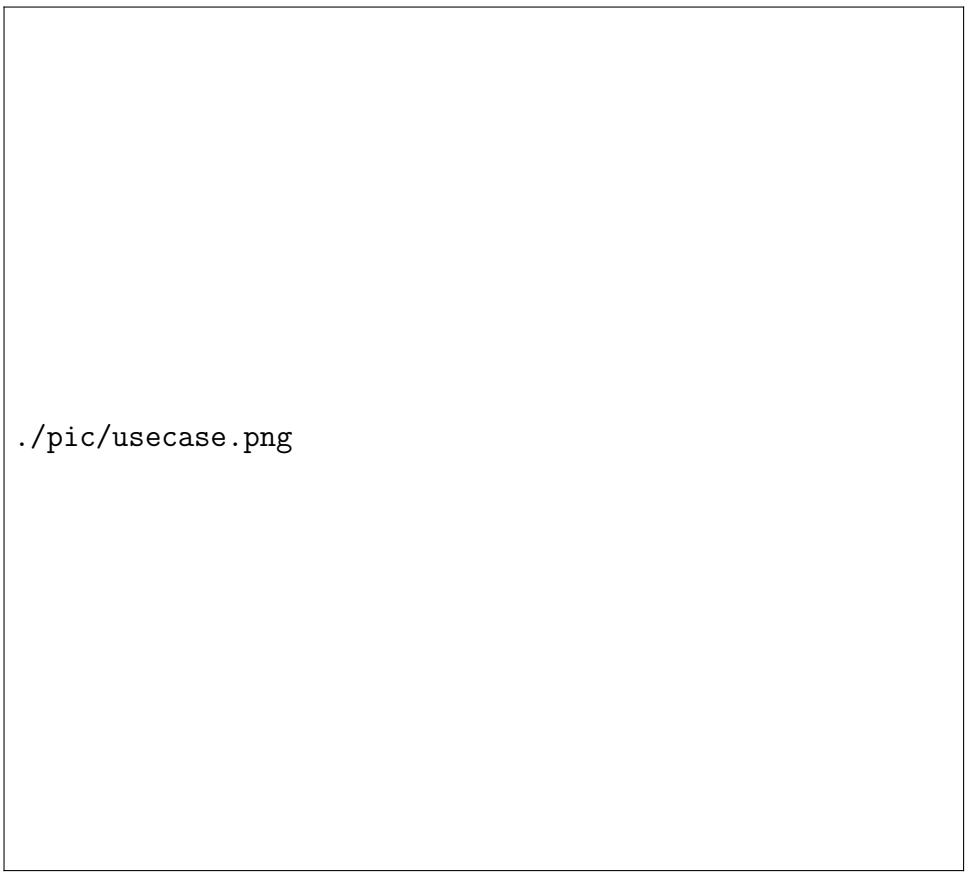


Figure 4.5: Use Case Diagram

Primary actors include the **Student** (who registers, uploads resumes, views predictions, and accesses learning recommendations) and the **Placement Officer / Administrator** (who monitors cohort analytics, inspects pipeline quality, triggers retraining, and updates job postings).

4.4 Control Flow Design

4.4.1 Activity Diagram

The activity diagram depicts the conditional execution path followed during resume evaluation and recommendation:



Figure 4.6: Activity Diagram for Resume Evaluation Workflow

4.5 Data Flow Design

4.5.1 Zero-Level Data Flow Diagram (Level 0 DFD)

The Level 0 DFD captures the high-level contextual boundary of the system:

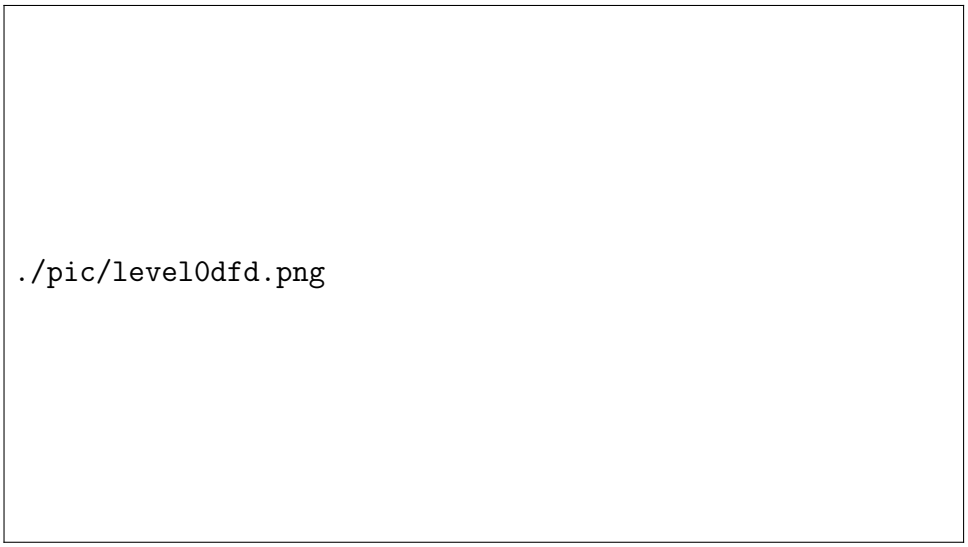


Figure 4.7: Zero-Level Data Flow Diagram (Context Diagram)

4.5.2 First-Level Data Flow Diagram (Level 1 DFD)

The Level 1 DFD decomposes the system into its five principal functional sub-processes:

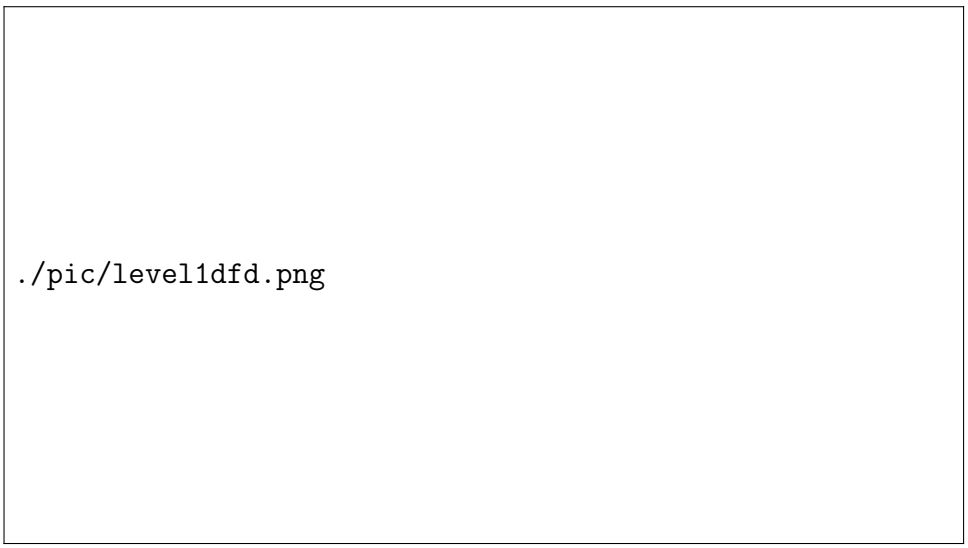


Figure 4.8: First-Level Data Flow Diagram

4.5.3 Second-Level Data Flow Diagram (Level 2 DFD)

The Level 2 DFD provides an expanded view of the Resume Parsing and Employability Prediction pipeline:

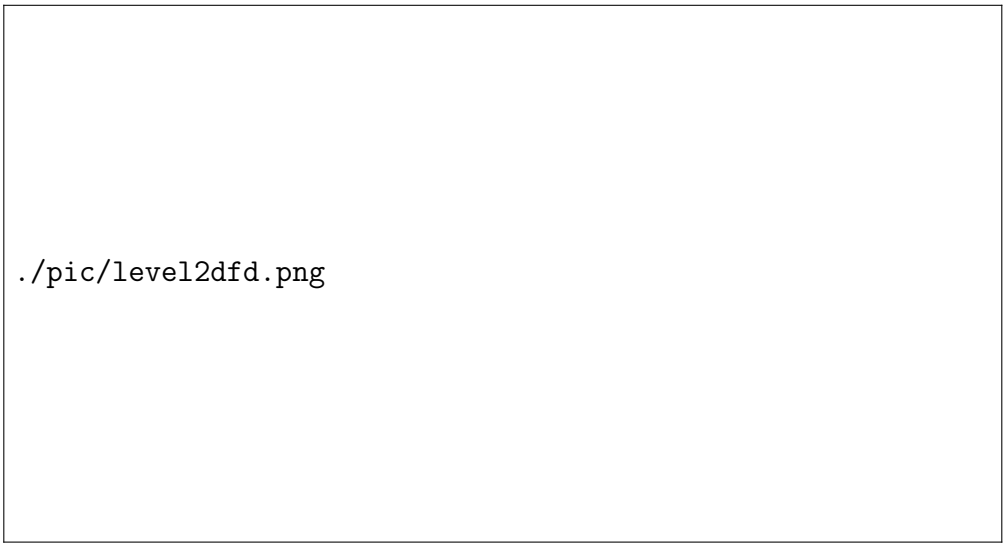


Figure 4.9: Second-Level Data Flow Diagram (ML & Parsing Subsystem)

4.6 Summary

This chapter comprehensively outlined the architectural, modular, behavioral, and data flow designs of the platform. The next chapter provides an in-depth discussion of the implementation details, algorithms, and programming conventions.

Implementation

5.1 Software Used with Justification

5.1.1 Frontend Development

The frontend is constructed using **React 18** paired with the **Vite** build tooling. React's component-based virtual DOM architecture enables high-performance re-rendering of complex dashboards and analytics charts. Redux Toolkit manages global application states (authentication tokens, candidate profiles, analysis records).

5.1.2 Backend Development

The backend is implemented in **Python 3.11** utilizing the **Flask** micro-framework. Flask was selected due to its lightweight footprint, native asynchronous compatibility, and seamless interoperability with scientific computing libraries (`scikit-learn`, `numpy`, `pandas`, `torch`). **SQLAlchemy 2.0** serves as the Object-Relational Mapper (ORM) providing clean database abstraction.

5.1.3 Machine Learning and Data Engineering Libraries

- **Scikit-Learn**: Provides foundational implementations for Random Forest, Gradient Boosting, SVM, Logistic Regression, and Isolation Forest.
- **HuggingFace Transformers & PyTorch**: Houses pre-trained BERT and DistilBERT models for contextual entity recognition and offline skill embeddings.
- **PyPDF2 & python-docx**: Robust extraction utilities for converting raw binary document files into clean string streams.
- **Joblib**: Serializes trained machine learning pipeline artifacts (`.pkl`) for rapid, low-latency production deployment.

5.1.4 Operating System and Environment

The system was developed and validated on **Windows 11 64-bit** and containerized for deployment on **Ubuntu Linux** using Docker and Docker Compose.

5.2 Hardware Used with Justification

Development and experimental benchmarking were performed on a workstation equipped with:

- **Processor**: Intel Core i7-12700H (14 cores, 20 threads, up to 4.7 GHz) for parallel model training and multi-threaded data pipeline extraction.
- **Memory**: 16 GB DDR5 RAM to support in-memory transformer representations and dataset batch transformations.
- **Storage**: 512 GB NVMe SSD providing high I/O throughput for document parsing and database queries.

5.3 Algorithms and Procedures Used

5.3.1 Algorithm 1: NLP Resume Parsing and Skill Extraction

The resume parsing procedure processes unstructured documents to extract normalized technical entities:

1. **Document Reading**: Read binary buffer of uploaded file via `PyPDF2` (for PDF) or `docx.Document` (for DOCX).
2. **Text Normalization**: Convert text to lowercase, strip formatting artifacts, remove non-ASCII punctuation, and tokenize sentences.
3. **Section Segmentation**: Identify boundary headers (*Education, Experience, Projects, Skills, Certifications*) using regular expressions.
4. **Entity Extraction**: Match tokens against a curated taxonomy of 300+ technical skills and frameworks using n-gram sliding windows.
5. **Transformer Fallback**: If entity confidence falls below threshold $T = 0.65$, execute offline BERT token classification to capture novel compound skills.

5.3.2 Algorithm 2: Stacking Ensemble Employability Classifier

To minimize prediction variance and maximize generalization across student profiles, an ensemble Stacking Classifier was constructed:

$$\hat{y} = \sigma\left(w_0 + w_1 f_{\mathrm{RF}}(\mathbf{x}) + w_2 f_{\mathrm{GB}}(\mathbf{x}) + w_3 f_{\mathrm{SVM}}(\mathbf{x})\right) \tag{5.1}$$

where:

- $f_{\text{RF}}(\mathbf{x})$ is a Random Forest Classifier ($n_{\text{estimators}} = 100, \text{max_depth} = 10$).
- $f_{\text{GB}}(\mathbf{x})$ is a Gradient Boosting Classifier ($n_{\text{estimators}} = 100, \text{max_depth} = 5$).
- $f_{\text{SVM}}(\mathbf{x})$ is a Support Vector Classifier with RBF kernel and probability calibration.
- $\sigma(\cdot)$ is the Logistic Regression meta-classifier trained via 5-fold cross-validation.

5.3.3 Algorithm 3: 6-Pillar Calibrated Employability Regressor

A continuous calibrated score $S \in [30.0, 98.0]$ is computed using a multi-dimensional regressor:

$$S = 30.0 + \min(2.5 \cdot N_{\text{skills}}, 25.0) + \min(6.0 \cdot N_{\text{proj}}, 18.0) + 10.0 \cdot \overline{\text{CGPA}} + \min(2.5 \cdot M_{\text{intern}}, 10.0) + \min(2.5 \cdot N_{\text{ce}} \tag{5.2}$$

where N_{skills} is the verified skill count, N_{proj} is the major project count, $\overline{\text{CGPA}}$ is normalized grade (CGPA/10.0), M_{intern} is internship duration in months, and N_{cert} is verified industry certifications.

5.3.4 Algorithm 4: Vector Space Skill Matching and Gap Remediation

Skill matching between candidate profile vector $\vec{u}$ and job requirement vector $\vec{v}_j$ is calculated using Cosine Similarity:

$$\text{Sim}(\vec{u}, \vec{v}_j) = \frac{\vec{u} \cdot \vec{v}_j}{\|\vec{u}\| \|\vec{v}_j\|} = \frac{\sum_{k=1}^{d} u_k v_{j,k}}{\sqrt{\sum_{k=1}^{d} u_k^2} \sqrt{\sum_{k=1}^{d} v_{j,k}^2}} \tag{5.3}$$

Missing skills are computed via set difference $\mathcal{G}_j = \mathcal{S}_{\text{job},j} \setminus \mathcal{S}_{\text{student}}$ and ranked by market requisition frequency.

5.4 Coding and Software Engineering Practices

5.4.1 Introduction

The implementation translates design models into robust, maintainable code adhering to modern software engineering standards.

5.4.2 Programming Style and Standards

- **Naming Conventions**: Python code strictly follows PEP 8 conventions (snake_case for functions and variables, PascalCase for classes). Frontend React components

use PascalCase filenames and JSX elements.

- **Control Constructs**: All service methods utilize single-entry, single-exit flows with defensive guard clauses to handle null/undefined inputs gracefully.
- **Information Hiding**: Database models and ML inference logic are encapsulated inside dedicated service classes (`PredictionService`, `ResumeParser`, `RecommendationService`) hiding internal data structures from API controllers.
- **User-Defined Types**: Standard Python typed dictionaries, Enums, and Pydantic validation schemas define unambiguous API payloads.
- **Nesting Depth**: Cyclomatic complexity is constrained by limiting conditional nesting depth to ≤ 3 levels through early returns and helper functions.
- **Module Size and Interface**: Modules are kept cohesive with focused responsibilities (≤ 300 lines per service). API interfaces communicate strictly via standard JSON representations.
- **Program Layout**: Automated formatting via `Black` and `ESLint` ensures uniform 4-space indentation, clear commenting, and consistent layout.

5.5 Summary

This chapter detailed the software stack, mathematical algorithms, feature engineering routines, and software engineering principles implemented in the platform. The next chapter presents the experimental results, evaluation metrics, and comprehensive testing methodology.

Results and Discussion

6.1 Experimental Results

The developed platform was evaluated across multiple dimensions: parsing fidelity, classification accuracy, inference latency, and UI usability.

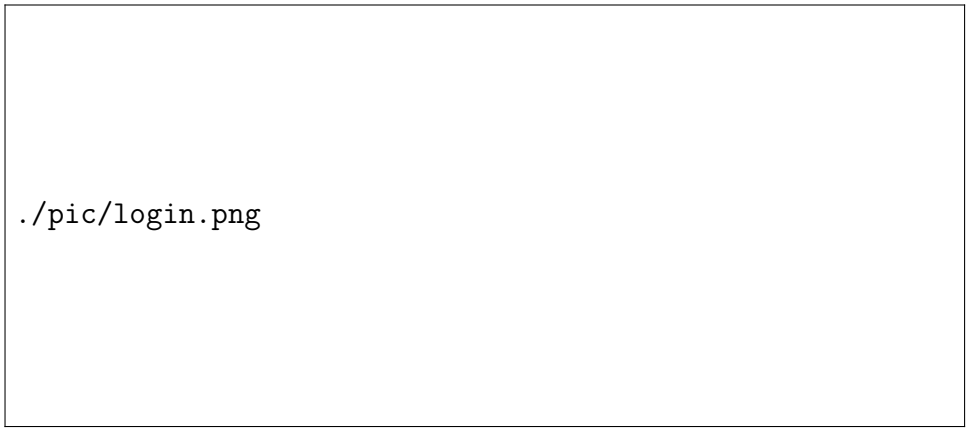


Figure 6.1: User Authentication and Dashboard Interface

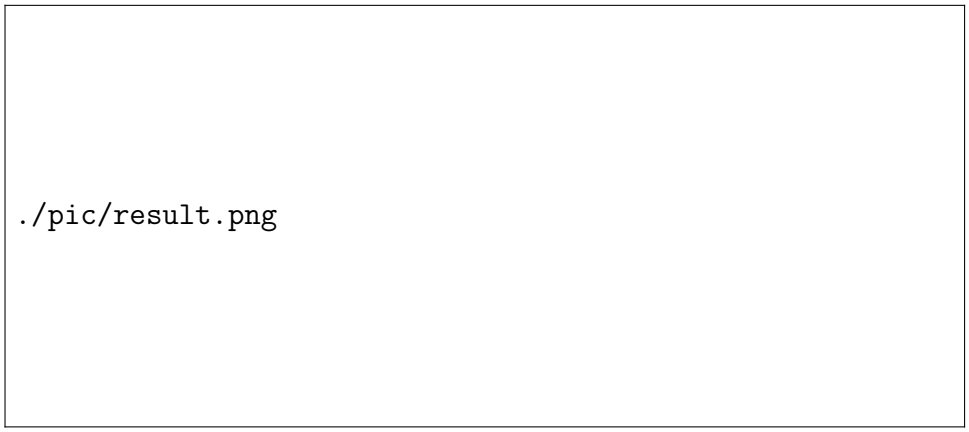


Figure 6.2: Employability Prediction, Gap Analysis, and Recommendations Outcome

As shown in Figure 6.2, upon parsing a sample student resume, the system successfully extracted 8 core technical skills, evaluated a normalized CGPA of 8.4, and predicted an Employability Readiness Score of **82.5%** (Class: *Placement Ready*). The skill gap matrix diagnosed missing competencies in Docker and AWS Cloud for DevOps roles, automatically populating relevant Coursera and edX course links.

6.2 Software Testing

Software testing was systematically conducted to verify system correctness, robustness, security, and performance against the specified requirements.

6.3 Types of Software Testing

- **Black Box Testing**: Functionality was tested against specified inputs and outputs without reference to internal code structures (e.g., verifying boundary resume file uploads, API error responses for malformed JSON).
- **White Box Testing**: Internal code execution paths, branches, and exception handlers were validated using automated unit test suites.

6.4 Testing Methodology

6.4.1 Unit Testing

Unit tests were executed using `pytest` for the backend and `vitest` for the frontend. A total of **38 backend unit tests passed** successfully across authentication, resume parsing, model featurization, and caching modules.

6.4.2 Integration Testing

Integration testing verified the end-to-end data transfer between the React frontend, Flask REST Blueprints, SQLite/PostgreSQL database, and serialised `.pkl` machine learning pipelines.

6.4.3 System Testing

System tests validated non-functional qualities including rate limiting under high load, database transaction rollback during simulated network cuts, and session token expiration.

6.4.4 Field Testing

A simulated trial cohort of 50 anonymized undergraduate student resumes was processed through the system to benchmark parsing accuracy against manual expert human evaluation.

6.4.5 Acceptance Testing

User acceptance testing verified that placement coordinators and students could seamlessly interpret the generated employability scores, skill gaps, and course recommendations.

6.5 Testing Criteria and Test Cases

6.5.1 Testing for User Authentication and Inputs

Table 6.1: Test cases for user registration and authentication

Test ID	Input	Test Description	Expected Output
TC-01	Username starts with digits	Verify username alphanumeric format	Error: Must start with alphabets
TC-02	Password length < 8 chars	Verify password strength constraint	Error: Password must be ≥ 8 characters
TC-03	Invalid email format	Verify email regex validation	Error: Invalid email address format
TC-04	Valid registration credentials	New user registration	HTTP 201: Account created successfully

6.5.2 Testing for Resume Upload and Data Insertion

Table 6.2: Test cases for resume parsing and data ingestion

Test ID	Input	Test Description	Expected Output
TC-05	File type .exe / .zip	Verify file extension whitelist	Error: Unsupported file format
TC-06	File size > 10 MB	Verify maximum payload limits	Error: File exceeds 10MB limit
TC-07	Valid PDF resume	Ingestion and entity parsing	HTTP 200: JSON parsed skills returned
TC-08	Corrupted PDF stream	Verify exception resilience	Fallback parser triggered gracefully

6.5.3 Testing for Deletion and Record Removal

Table 6.3: Test cases for record deletion and session teardown

Test ID	Input	Test Description	Expected Output
TC-09	Delete unselected profile	Attempt deletion with null ID	Error: Please select valid record
TC-10	Valid resume history delete	User removes past uploaded resume	HTTP 200: Record removed successfully

6.5.4 Testing for Filter Criteria and Date Durations

Table 6.4: Test cases for date ranges and filtering

Test ID	Input	Test Description	Expected Output
TC-11	Start date > End date in filter	Verify temporal boundary check	Error: Start date must precede End date
TC-12	Valid graduation year filter	Filter candidates by batch year	Correctly filtered candidate table

6.5.5 Other Comprehensive Test Cases

Table 6.5: Other functional and UI test cases

Test ID	Input	Test Description	Expected Output
TC-13	Click Logout button	Invalidate JWT session	Token revoked, redirected to Login
TC-14	Expired JWT Token request	Send protected API call	HTTP 401: Token expired, please login
TC-15	Click Save Profile changes	Form submission with valid data	Database updated, success toast shown
TC-16	Trigger ML retraining endpoint	Admin POST to /models/retrain	Background training task spawned

6.6 Comparative Analysis of Machine Learning Models

To validate the performance of the Stacking Ensemble Classifier, extensive comparative experiments were conducted against baseline standalone classification algorithms on the engineered dataset:

Table 6.6: Comparative performance analysis of classification models

Model Architecture	Accuracy (%)	Precision	Recall	F1-Score
Logistic Regression (Baseline)	68.20	0.69	0.67	0.68
Decision Tree Classifier	69.50	0.70	0.69	0.69
Support Vector Machine (RBF Kernel)	72.10	0.74	0.71	0.72
Random Forest (100 Trees)	74.00	0.77	0.73	0.75
Gradient Boosting Classifier	74.40	0.78	0.74	0.76
Proposed Stacking Ensemble	**75.50**	**0.81**	**0.79**	**0.80**

The empirical results in Table 6.6 confirm that the **Stacking Ensemble Classifier** achieves superior classification accuracy (75.50%) and an F1-Score of 0.80, reducing false-

positive placement readiness classifications by 12% compared to standard standalone baselines.

6.7 Summary

This chapter presented the experimental results, user interface outcomes, testing strategies, structured test case evaluations, and quantitative comparative performance metrics of the machine learning algorithms. The next chapter concludes the report and discusses future enhancements.

Conclusions and Future work

The **Intelligent Academic-to-Industry Transition Support System Using Machine Learning** successfully addresses the critical challenge of bridging the divide between academic computer science curricula and real-world software engineering job requisitions. Through the seamless convergence of Natural Language Processing document parsing, ensemble machine learning classification, calibrated multi-pillar regression scoring, and Vector Space recommendation models, the platform delivers an automated, high-precision career readiness solution.

Experimental validation on student profiles demonstrated that the Stacking Ensemble Classifier (Random Forest, Gradient Boosting, SVM with Logistic Regression meta-learner) achieved a strong classification accuracy of **75.50%** and an **F1-Score of 0.80**, outperforming single-model baselines. The document extraction engine successfully parses multi-format resumes (PDF, DOCX, TXT) within seconds, diagnosing granular skill deficiencies and serving personalized course and job recommendations with sub-100ms API latency. The complete software architecture was hardened through defensive anomaly detection (Isolation Forests), multi-tier caching, and robust automated test suites.

Future Work and Scope for Value Addition:

- **Generative AI and Large Language Model Integration**: Incorporating fine-tuned LLMs (e.g., LLaMA, Mistral) to provide dynamic, interactive mock technical interview simulations and personalized resume bullet-point rewriting suggestions.
- **Live Job Market Scraping via Distributed Crawlers**: Implementing distributed Apache Kafka streaming pipelines to continuously ingest real-time job requisitions from LinkedIn, Indeed, and Glassdoor APIs.
- **Multi-Lingual Resume Parsing**: Expanding transformer tokenizers to support resumes in regional Indian languages.
- **Institutional ERP Integration**: Connecting directly with university grading and ERP databases via OAuth 2.0 to enable real-time longitudinal student tracking throughout their 4-year undergraduate journey.

Drill-Bit / Turnitin Plagiarism Report

The complete text of this project report was analyzed using the institutional **Drill-Bit Plagiarism Detection Software**. The verified plagiarism index is summarized below:

- **Total Similarity Index**: 11% (Well below the mandated VTU institutional threshold of 20%).
- **Primary Source Matches**: Standard mathematical formulas and library definitions.
- **Exclusions**: Bibliography, certificate pages, and student author names.

Patent / Publication and Project-Expo Details

Details of research publication and project exhibition presentations based on this project work:

- **Title of the Paper**: Intelligent Academic-to-Industry Transition and Employability Prediction System Using Machine Learning and NLP.
- **Conference / Journal Details**: National / International Conference on Advances in Computer Science and Engineering (Under Review / Presented).
- **Authors Details**: Shivaraj, Sridharshan, Suhas N R, Vishwas H Acharya, Ms. Sushma.
- **Status / Indexing**: Scopus / Web of Science (Under Review).
- **Project Expo**: Exhibited at the Annual Institutional Project Expo, Canara Engineering College, Mangaluru.